



GitHub Trending Top 10

每日热门开源项目深度解读

2026-09-25

今日榜单

1	paperclipai/paperclip	TypeScript	NEW
The open-source app everyone uses to manage agents at work			
2	anthropics/claude-plugins-official	Python	NEW
Official, Anthropic-managed directory of high quality Claude Code Plugins.			
3	vectorize-io/hindsight	Python	2天
Hindsight: Agent Memory That Learns			
4	obra/superpowers	Shell	3天
An agentic skills framework & software development methodology that works.			
5	mattpocock/skills	Shell	NEW
Skills for Real Engineers. Straight from my .agents directory.			
6	dream-num/univer	TypeScript	4天
The Office Harness for AI Agents — Spreadsheets, Docs, Slides, Canvas, Relationa...			
7	anthropics/skills	Python	NEW
Public repository for Agent Skills			
8	androoAGI/starnet	JavaScript	NEW
A living pixel-art station where real AI agents do real work. Local-first deskto...			
9	derv82/wifit3	Python	NEW
Wifite but USB-only & cross-platform.			
10	kelseyhightower/kubernetes-the-hard-way	Unknown	NEW
Bootstrap Kubernetes the hard way. No scripts.			

#1

TypeScript

NEW

paperclip

The open-source app everyone uses to manage agents at work

Stars **83,810** Forks **15,160** Today **+1,853**

<https://github.com/paperclipai/paperclip>

项目简介：Paperclip 是一个开源的 AI Agent 编排平台，定位是「管理 AI 员工的公司级系统」——如果说 OpenClaw 是员工，Paperclip 就是公司。它解决的核心问题是：当团队同时运行大量异构 AI Agent（Claude Code、Codex、Cursor 等）时，如何统一分配目标、追踪进度、控制成本，而不是在几十个终端窗口间手忙脚乱。

核心功能：

- **目标驱动的任务管理：**以业务目标（如"做到 100 万美元 MRR"）为起点，而非 PR 或工单，Agent 自动拆解执行，人类在关键节点审批。
- **多 Agent 编排与组织架构：**支持雇佣任意提供商的 Agent 担任 CEO、CTO、工程师等角色，构建虚拟组织架构并协调协作。
- **成本与预算治理：**内置预算控制和成本监控，防止自主 Agent 无限制消耗 API 费用。
- **审计与人工介入：**所有 Agent 工作可追溯（diff、截图等），支持 24/7 自主运行的同时随时人工干预。
- **移动端管理：**可从手机监控和指挥整个自主业务运行。

适用场景：适合已经在用多个 AI 编码/执行 Agent 的重度用户（比如同时开 20 个 Claude Code 终端的开发者）、想要构建"自主 AI 组织"的创业者，以及需要为 Agent 团队建立预算和治理流程的团队。典型场景是用 Agent 群组自动化运营一个小型业务或产品线。

技术亮点：采用 Node.js 服务端 + React UI 的经典全栈架构，用 TypeScript 编写。设计上抽象了"心跳（heartbeat）"协议——任何能接收心跳的 Agent 都能被纳入编排，这意味着它对底层 Agent 提供商高度解耦，不绑定特定模型或工具。四大支柱（任务、组织、训练、基础设施）的架构划分也体现了从"工具"到"组织系统"的思路升级。

上手建议：从 README 的 Quickstart 和官方文档（docs.papercliping）入手，先跑通一个最小目标 + 单 Agent 的流程，再逐步扩展到多角色团队。贡献方面建议先加入 Discord 社区了解开发方向，关注其 Agent 接入协议（心跳机制）相关的 issue，这是项目扩展性的关键接口。

#2

Python

NEW

claude-plugins-official

Official, Anthropic-managed directory of high quality Claude Code Plugins.

Stars **36,783** Forks **4,149** Today **+62**

<https://github.com/anthropics/claude-plugins-official>

项目简介：这是 Anthropic 官方维护的 Claude Code 插件目录（Marketplace），为 Claude Code 用户提供经过筛选的高质量插件分发渠道。它解决的问题是：在插件生态野蛮生长时，用户缺乏一个可信、规范的插件来源，而此项目通过官方审核和标准化结构，降低了发现与安装插件的门槛。

核心功能：1. **插件市场与一键安装：**用户可通过 `/plugin install {name}@claude-plugins-official` 或 `/plugin > Discover` 直接浏览和安装插件。2. **内外部插件分区管理：**`/plugins` 存放 Anthropic 官方插件，`/external_plugins` 收纳第三方合作伙伴与社区插件。3. **标准化插件结构：**统一约定 `plugin.json`、MCP 配置、`commands`、`agents`、`skills` 等目录规范。4. **不可变命名与重命名迁移：**插件 slug 一旦发布不可更改，通过 `renames` 映射实现旧安装的自动迁移。5. **Skill-bundle 支持：**允许无 manifest 的仓库通过 `strict: false` 直接声明 `skills` 数组，灵活接入现有 SDK。

适用场景：Claude Code 的日常用户希望快速扩展其能力（如接入 MCP 服务器、自定义斜杠命令、Agent 与 Skill）；插件开发者（尤其是第三方合作伙伴）希望将自己的工具提交到官方目录获得曝光；企业内部团队也可参考其结构自建私有插件市场。

技术亮点：一是**命名不可变 + renames 自动迁移**机制，在插件加载器层面透明重写旧 slug，避免用户因重命名而 `plugin-not-found`，体现了对向后兼容的工程考量；二是 **skill-bundle 的 strict: false 模式**，让没有 `.claude-plugin/plugin.json` 的现有仓库也能被市场收录，通过相对路径暴露多层级 `skills`，降低了接入成本；三是 MCP 与插件体系深度整合，插件可携带自己的 MCP server 配置。

上手建议：使用者直接在 Claude Code 中运行 `/plugin install` 或浏览 Discover 即可。贡献者应先阅读 `/plugins/example-plugin` 参考实现，按标准结构组织代码，第三方插件需通过官方提交表单并满足质量与安全标准。开发者务必注意 `name` 字段的不可变性，UI 展示应改用 `displayName`，确需重命名时在 `marketplace.json` 中补充 `renames` 映射。

#3

Python

连续 2 天

hindsight

Hindsight: Agent Memory That Learns

Stars **28,738** Forks **2,895** Today **+1,652**

<https://github.com/vectorize-io/hindsight>

项目简介：Hindsight 是由 Vectorize.io 推出的 Agent 记忆系统，目标不是让 AI 简单地“记住”对话历史，而是让 Agent 能够像人一样从经验中“学习”。它针对 RAG 和知识图谱在长期记忆任务上的不足进行改进，宣称在 LongMemEval 基准测试中达到 SOTA 水平。

核心功能：1. **三大操作原语：**retain（保留）、recall（召回）、reflect（反思），覆盖记忆的写入、检索与抽象提炼全流程。2. **记忆类型与观察机制：**支持多种记忆类型，并通过 observations（观察）和 mental models / knowledge pages（心智模型与知识页）对记忆进行结构化沉淀。3. **多形态接入：**提供 Docker 服务端、Python 嵌入式模式（无需服务器）、MCP Server，以及 LLM Wrapper（两行代码接入）和多种平台集成。4. **多 LLM 提供商兼容：**支持 25+ 提供商，包括 OpenAI、Anthropic、Gemini、Ollama 本地模型等。5. **生产级部署：**支持 memory banks 隔离、云端托管方案，已在财富 500 强企业生产环境使用。

适用场景：- 构建需要长期记忆的对话式 AI Agent（客服、个人助手、教育辅导等）。- 需要 Agent 跨会话积累经验并持续改进的自动化工作流。- 使用 Claude Code、Cursor 等编码助手时，通过 skill 快速查阅文档辅助开发。- 对记忆准确率有要求、且需要可复现基准数据的企业级应用。

技术亮点：1. **基准表现经独立验证：**LongMemEval 成绩由弗吉尼亚理工 Sanghani 中心和《华盛顿邮报》复现，而非仅靠厂商自报，可信度较高。2. **超越传统 RAG/知识图谱：**明确提出并解决 RAG 与知识图谱在长期记忆上的短板，强调“学习”而非“检索”。3. **嵌入式模式：**Python 可直接内嵌运行，无需单独部署服务，降低集成门槛。4. **学术支撑：**附带 arXiv 论文（2512.12818），技术路线有理论依据。

上手建议：- 最快路径：用 Docker 一行命令启动服务端，配置 OPENAI_API_KEY 后访问 UI（:9999）体验。- 轻量集成：使用 LLM Wrapper 两行代码接入现有 Agent，或采用 Python 嵌入式模式免部署。- 深入理解：先读 Core Concepts 中的 memory types 和 retain/recall/reflect 三操作，再看 Cookbook 示例。- 贡献方向：项目为 MIT 协议，可从 integrations 扩展、benchmark 复现或文档完善入手，Slack 社区可获取支持。

#4

Shell

连续 3 天

superpowers

An agentic skills framework & software development methodology that works.

Stars **291,483** Forks **26,080** Today **+465**

<https://github.com/obra/superpowers>

项目简介：Superpowers 是一套面向编码智能体（coding agent）的完整软件开发方法论，由一系列可组合的"技能"（skills）和初始指令构成。它解决的核心问题是：让 AI 编码助手不再一上来就埋头写代码，而是先理解需求、产出规格、制定计划，再有序执行。

核心功能：

- **需求澄清与规格拆解：**智能体启动后会先反问用户真正想做什么，把需求整理成规格，并以易于消化的小块呈现给用户确认。
- **实现计划生成：**基于确认后的设计，生成详细到"能力平庸的初级工程师也能照做"的实施计划，强调 TDD、YAGNI、DRY 原则。
- **子智能体驱动开发：**用户说"go"之后，系统启动 subagent-driven-development 流程，让多个智能体逐项完成任务并互相审查，可自主工作数小时而不偏离计划。
- **自动触发：**技能会自动激活，用户无需额外操作，编码智能体"天然"具备这套能力。
- **多平台适配：**支持 Claude Code、Codex、Cursor、Gemini CLI、Copilot CLI 等十余种主流编码智能体环境。

适用场景：适合所有使用 AI 编码智能体的开发者，尤其是希望让 AI 承担较复杂、多步骤开发任务（而非零散代码片段）的团队或个人。在企业环境中还可通过商业支持服务获得托管与额外工具链。

技术亮点：项目用 Shell 编写，本质是一套"方法论即插件"的设计——通过各平台的插件/扩展机制注入统一的技能体系与 session-start 钩子。它把软件工程的最佳实践（TDD、YAGNI、DRY、规格先行）编码成智能体可自动执行的流程，并用子智能体分工+互审的方式模拟真实团队协作，这是它区别于普通 prompt 模板的关键。

上手建议：先从你常用的编码智能体对应的安装章节入手（如 Claude Code 用 `/plugin install superpowers@claude-plugins-official`），装好后按 README 的 "The Basic Workflow" 走一遍完整流程。想深入贡献可阅读 "What's Inside" 和 "Philosophy" 了解技能组合方式，再参考 "Contributing" 提交新技能或改进。

#5

Shell

NEW

skills

Skills for Real Engineers. Straight from my .agents directory.

Stars **269,492** Forks **22,715** Today **+671**

<https://github.com/mattpocock/skills>

项目简介：这是 Matt Pocock（TypeScript 社区知名教育者）开源的 AI Agent 技能集合，直接来自他个人的 .agents 目录。它解决的核心问题是：当前 AI 编码工具（如 Claude Code、Codex）虽然强大，但容易“跑偏”——要么没理解需求就动手，要么输出啰嗦失控。这套技能用工程化方式约束和引导 Agent 行为，定位是“真正的工程实践，而非 vibe coding”。

核心功能：最受欢迎的是 /grill-me 和 /grill-with-docs——在动手前让 Agent 反过来“拷问”你，逼你把需求想清楚，从根源上解决人机对齐问题；/setup-matt-pocock-skills 用于初始化配置（选择 issue tracker、标签、文档路径）；/triage 结合标签做任务分类；整套技能以可组合的小模块形式提供，支持任意模型。

适用场景：主要面向使用 Claude Code、Codex 等 AI 编码 Agent 的开发者，尤其是做真实项目（而非玩具 demo）的工程师。当你发现 Agent 总是“答非所问”、输出冗长、或需要一套可复用的 workflow 来规范 AI 协作时，这套技能就很合适。

技术亮点：设计哲学上明确反对 GSD、BMAD、Spec-Kit 这类“接管整个流程”的重型框架，主张小而可改、可组合、不夺走开发者控制权。安装方式提供了两种哲学选择：Claude Code 插件是“订阅式”只读更新，skills.sh 则是“复制到你仓库”的可编辑文件——把“要不要 fork”这个决策显式交给用户。技能本身用 Shell 组织，基于数十年工程经验沉淀。

上手建议：从 `npx skills@latest add mattpocock/skills` 开始，务必勾选 `setup-matt-pocock-skills`，然后在仓库里运行一次 `/setup-matt-pocock-skills` 完成配置。建议先只装 /grill-me 试用，感受“被 Agent 反向提问”的体验；想深入贡献可参考 .agents/adr/ 目录下的架构决策记录，了解项目的演进思路。

#6

TypeScript

连续 4 天

univer

The Office Harness for AI Agents — Spreadsheets, Docs, Slides, Canvas, Relational Tables, and PDF in one runtime.

Stars **18,186** Forks **1,553** Today **+1,048**

<https://github.com/dream-num/univer>

项目简介：Univer 是一个开源的 Office 办公套件 SDK，用 TypeScript 编写，目标是让开发者把电子表格、文档、演示文稿、看板、关系表等能力嵌入到自己的产品中，而不是被迫使用某个固定的在线办公应用。它解决的核心问题是：企业或开发者想构建自己的类 Excel/Word 体验时，往往需要从零造轮子，而 Univer 提供了可插拔、可定制完整底层框架。

核心功能：一是多品类 Office 能力，覆盖电子表格、文档、幻灯片、Canvas、关系表，PDF 也在规划中；二是插件化架构，开发者可以按需组合功能，也可以基于预设快速启动；三是统一的 Facade API，同一套接口在浏览器和 Node.js 服务端都能运行；四是内置公式引擎和基于 Canvas 的渲染，保证高性能与可定制性；五是支持协作与 AI Agent 在同一文件中工作，内容可跨工具引用和联动。

适用场景：适合需要在 SaaS 产品、内部工具、BI workflow 或 AI 应用中嵌入表格/文档编辑能力的团队；也适合希望在服务端处理工作簿和文档的场景。典型用户包括做低代码平台、数据看板、在线协作工具、AI 办公助手的开发者，以及需要自托管办公套件的企业。

技术亮点：采用 Canvas 渲染而非 DOM，在高数据量下性能优势明显；插件架构让功能解耦，便于裁剪和扩展；浏览器与 Node.js 共用同一套架构，降低了前后端能力复用的成本；公式引擎和 Facade API 的设计，使得办公能力可以像积木一样被组合进 AI 应用，契合当前 AI Agent 操作办公文件的趋势。

上手建议：建议先从官网文档和 Showcase 入手，跑通一个最小嵌入示例，理解 Facade API 和插件机制；再参考 Univer Workspace 这个开源自托管项目，学习完整集成方式。想贡献代码的话，可以从插件、命令、UI 组件或文档翻译切入，项目对多语言文档和社区生态比较友好，Discord 和 GitHub Issues 是主要的交流渠道。

#7

Python

NEW

skills

Public repository for Agent Skills

Stars **178,120** Forks **21,100** Today **+155**

<https://github.com/anthropics/skills>

项目简介：这是 Anthropic 官方开源的 Claude Agent Skills 仓库，本质上是一套"给 AI 装技能包"的标准与实践集合。它解决的核心问题是：如何让 Claude 在特定专业任务（如生成带品牌规范的文档、按企业流程分析数据）上表现得可复用、可定制，而不是每次从零对话。

核心功能：

- **技能包机制：**每个技能是一个独立文件夹，内含 SKILL.md（YAML frontmatter + 指令）及配套脚本/资源，Claude 按需动态加载。
- **官方示例技能集：**覆盖创意设计、开发技术、企业沟通、文档处理四大类，供开发者参考借鉴。
- **生产级文档技能：**docx/pdf/pptx/xlsx 四个技能正是 Claude 文档能力背后的实现，以 source-available 形式开放作参考。
- **规范与模板：**提供 Agent Skills 规范 (/spec) 和技能模板 (/template)，降低自定义门槛。
- **多端接入：**支持 Claude Code（插件市场）、Claude.ai（付费计划）、Claude API 三种使用方式。

适用场景：企业开发者想为内部工作流（品牌规范、数据流程、自动化任务）定制 Claude 能力时会用到；AI 工程师研究如何设计高质量 Agent 指令与工具调用模式时，可将其作为参考范本；普通 Claude 付费用户则可直接调用现成示例技能完成文档处理等任务。

技术亮点：设计上极简——一个文件夹 + 一个 SKILL.md 即可定义技能，用 YAML frontmatter 声明 name 和 description 作为元数据，Markdown 正文承载指令。这种"约定优于配置"的思路让技能易于版本管理、分发和组合，同时官方把生产环境实际使用的文档技能开源出来，是研究真实 AI 产品工程实现的难得样本。

上手建议：想快速体验，在 Claude Code 中执行 `/plugin marketplace add anthropics/skills` 后安装 `document-skills` 或 `example-skills` 即可；想创建自己的技能，从 `/template` 复制模板，参照 `/spec` 规范填写 SKILL.md；想深入理解复杂技能设计，重点研读 `skills/docx`、`skills/pdf` 等生产级示例。注意官方声明这些技能仅供演示与学习，实际行为可能与线上 Claude 有差异，用于关键任务前务必自行充分测试。

#8

JavaScript

NEW

starnet

A living pixel-art station where real AI agents do real work. Local-first desktop agent harness - bring your own key, watch your crew actually run.

Stars **330** Forks **77** Today **+113**

<https://github.com/androoAGI/starnet>

项目简介：StarNet 是一个"本地优先"的桌面 AI 智能体运行框架，把多个 AI Agent 组织成一个像素风格的太空站里，让它们在真实模型和真实工具下干活。它要解决的问题是：让多智能体协作不再是黑箱模拟，而是可观察、可审计、有真实成本记录的生产力工具。

核心功能：一是"团队而非聊天机器人"——可创建多个有独立人格、权限、工作区的 Agent 并发运行；二是自带密钥接入模型（OpenRouter、Anthropic、OpenAI、Google 等），密钥存在系统钥匙串；三是通过 Telegram、Discord、Slack、Signal 等远程指挥 Agent；四是"夜班模式"，离开时 Agent 在明确授权范围内继续工作并留痕；五是 OUTBOX 交付真实文件、MCP 连接器扩展能力。

适用场景：适合希望把 AI Agent 真正用于日常自动化任务的开发者、独立创作者和小团队——比如让 Agent 定时跑多步流程、在多个渠道接收任务、产出可打开的真实文件，而不是停留在对话框里。也适合对"AI 干活是否真实"有强审计需求、重视数据本地的用户。

技术亮点：最独特的是"空间即工作流"的设计哲学——房间=能力受限的团队、走廊=授权交接通道、摆放的物体=真实权限授予，界面布局直接映射运行时的权限与状态。其次是严格的"产品契约"：界面绝不展示 harness 无法证明的状态，不模拟收入、花费或完成度，模型调用、工具执行、开销账本全部落盘可查。

上手建议：普通用户直接从 releases 页面下载 Windows 或 macOS 安装包（Apple Silicon 务必选 aarch64 版本），填入自己的模型密钥即可开始。开发者可从源码运行，先读 docs/INDEX.md 和 INSTALL.md 了解架构与安装校验细节，贡献前建议先看 CONTRIBUTING.md。项目处于早期阶段，Windows 支持最成熟，macOS 覆盖较少，遇到问题可直接邮件反馈。

#9

Python

NEW

wifit3

Wifite but USB-only & cross-platform.

Stars **743** Forks **76** Today **+168**

<https://github.com/derv82/wifit3>

wifit3 项目解读

项目简介

wifit3 是经典无线审计工具 Wifite 的重写版本，定位为"USB 专用、跨平台"的独立 Wi-Fi 审计工具。它通过内置的纯 Python 无线协议栈直接驱动 USB 网卡，绕开了 Linux 内核驱动版本兼容和 Windows NDIS 的痛点，让同一套工具在三大操作系统上表现一致。

核心功能

- **多网卡协同侦察**：支持同时使用多张网卡抓包，并指定专用网卡进行注入，配合 2.4/5GHz 跳频扫描和实时数据包仪表盘。
- **WPA/WPA2 攻击套件**：涵盖握手包抓取、去认证攻击、PMKID 主动/被动采集，并导出 .pcap 与 .hc22000 格式。
- **WPS 恢复三件套**：PixieDust 离线 PIN 破解（利用 PRNG 弱点）、PushButton 按键监听直接提取明文 PSK、可断点续跑的 PIN 暴力破解。
- **WPA3 降级 EvilTwin**：克隆目标 AP 并通过 CSA/BTM/去认证驱逐客户端，实现 WPA3 到 WPA2 的降级握手捕获。
- **WEP 完整套件**：纯 Python 实现 ARP 重放、ChopChop、假认证和 PTW 密钥恢复。

适用场景

主要面向**无线安全研究人员、渗透测试工程师和 CTF 玩家**，用于授权范围内的 Wi-Fi 安全评估。由于支持 Windows/macOS，也适合那些无法使用 Kali Linux 但需要在其他平台上做无线审计的用户。普通用户如果想测试自家路由器安全性（如 WPS 是否开启、密码强度）同样适用。

技术亮点

最值得关注的是**"零运行时依赖 + 内置无线栈"**的设计：不依赖 aircrack-ng、reaver 等外部工具，直接用 PyUSB 操作 USB 网卡、用 Textual 构建 TUI 界面，通过 PyInstaller 打包成单文件可执行程序。这意味着用户不需要折腾驱动编译或依赖环境，插上受支持的网卡（如 ALFA 系列、Panda 系列）即可运行，跨平台一致性极高。

上手建议

使用者：先确认自己拥有受支持的 USB 网卡（尤其是 ALFA AWUS036 系列或 Panda 系列），然后从 Releases 页面下载对应平台的预编译二进制，首次运行会通过 pkexec/UAC 完成一次性驱动配置。**贡献者：**从 README 中的"Run from Source"入手，用 uv sync 搭建环境，重点可关注 docs/SUPPORTED-HARDWARE.md 中芯片支持列表的扩展，以及新芯片驱动的适配——这是项目最核心也最需要社区贡献的部分。

#10

Unknown

NEW

kubernetes-the-hard-way

Bootstrap Kubernetes the hard way. No scripts.

Stars **50,040** Forks **15,910** Today **+105**

<https://github.com/kelseyhightower/kubernetes-the-hard-way>

项目简介：这是 Kelsey Hightower 编写的经典 Kubernetes 教程，手把手教你在没有自动化脚本的情况下从零搭建一个 Kubernetes 集群。它不解决"快速部署"的问题，而是解决"真正理解 K8s 内部机制"的问题——通过走最长的路，让你搞懂启动一个集群到底需要哪些步骤。

核心功能：① 从证书签发（CA、TLS）开始，手动生成所有组件所需的证书和密钥；② 手动生成 kubeconfig 认证配置文件与数据加密密钥；③ 逐台引导搭建 etcd 集群、控制平面（API Server、Controller Manager、Scheduler）和 Worker 节点；④ 配置 Pod 网络路由并完成冒烟测试与清理。每个 Lab 都对应一个核心组件的完整引导流程。

适用场景：适合已经会用 kubeadm 或托管服务、但想深入理解 K8s 各组件如何协同工作的工程师；也适合准备 CKA/CKS 认证、需要吃透底层原理的人。不适合想快速上生产集群的团队——作者本人也强调结果不适用于生产环境。

技术亮点：最大价值在于"去魔法化"——把 kubeadm 等工具隐藏的证书体系、认证链路、组件启动参数全部摊开。项目紧跟上游版本（当前 K8s v1.32、containerd v2.1、etcd v3.6），并已支持 ARM64，反映了云原生工具链的最新实践。

上手建议：先准备好 4 台同网段的 ARM64/AMD64 机器（虚拟机即可），从 docs/01-prerequisites.md 开始按顺序做，不要跳步。想贡献的话，可以关注文档更新以跟进上游版本变化，或在 Issues 中反馈步骤中遇到的坑——由于是教学性质，社区支持有限，动手实践比提 PR 更符合项目初衷。

数据来源: github.com/trending

报告日期: 2026-09-25

由 DeepSeek AI 提供智能分析 | 自动生成